

Class 7: Machine Learning 1

Hailey Phan (PID: A19113445)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	8
Principal Component Analysis (PCA)	10
Analysis of UK Food Data	10
Data Import	10
Exploratory analysis	13
Tidy the data	14
PCA to the rescue	18

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

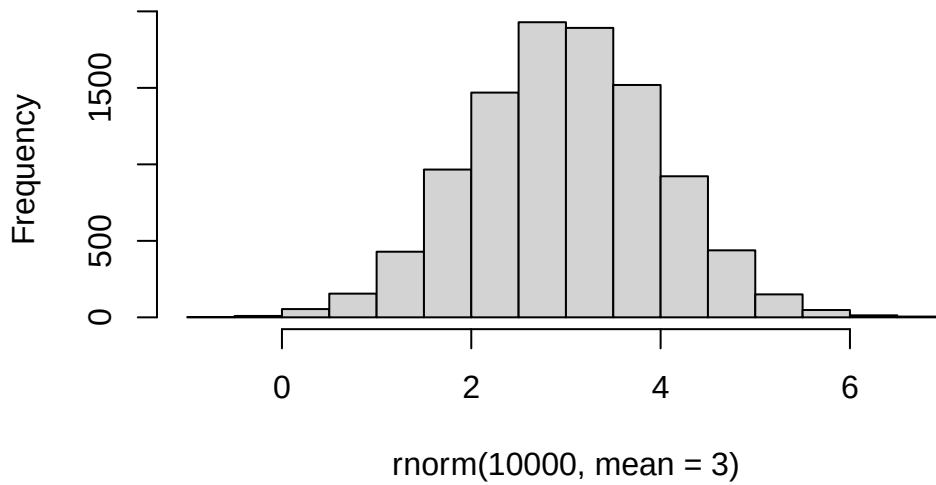
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`.

Before we go too deep, let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To help us do this, we can use the `rnorm()` function:

```
hist(rnorm(10000, mean = 3))
```

Histogram of rnorm(10000, mean = 3)



```
rnorm(30, -3)
```

```
[1] -2.666852 -4.884367 -3.227785 -4.570775 -1.885950 -2.969709 -2.275172  
[8] -3.973493 -2.743976 -2.552186 -3.158693 -2.703799 -4.063439 -5.356496  
[15] -4.396282 -4.118329 -3.300467 -1.577841 -2.696640 -2.363368 -3.162308  
[22] -4.562873 -4.215788 -2.803725 -2.001297 -3.433935 -4.562605 -3.612090  
[29] -2.448288 -2.627499
```

```
rnorm(30, +3)
```

```
[1] 3.3388574 2.4823724 2.7697613 4.0089567 4.3431779 3.3780656 3.0972786  
[8] 2.0806901 2.5364995 3.3065663 2.9293868 4.1368227 3.6666466 4.3293830  
[15] 2.6220948 0.9107120 2.4455526 3.9314520 2.8349151 2.5581322 3.7033036  
[22] 5.1708840 3.7396270 1.8886342 2.6669669 3.1810062 0.7202641 2.7724715  
[29] 2.8663108 3.5461919
```

```
x <- c(rnorm(30, -3), rnorm(30, +3))  
x
```

```
[1] -4.798079 -1.081840 -2.255255 -1.175925 -1.524991 -4.798187 -3.153930  
[8] -1.160772 -2.966038 -3.088822 -4.004282 -3.077800 -3.410450 -2.746674
```

```

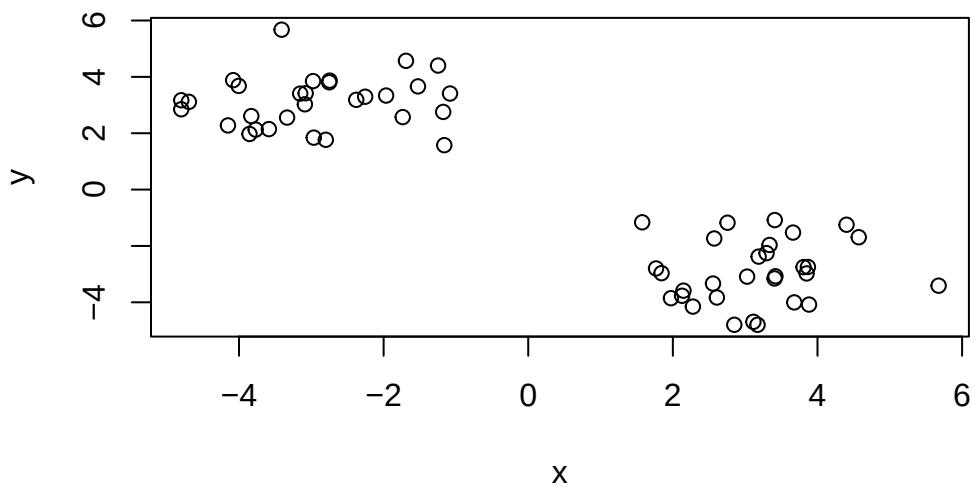
[15] -1.735809 -1.690726 -2.752084 -3.829838 -2.975825 -3.334764 -4.152640
[22] -1.965783 -2.378798 -3.768687 -3.586461 -1.247054 -3.854697 -4.080677
[29] -4.692990 -2.799249  1.767665  3.113654  3.883207  1.973583  4.401490
[36]  2.146612  2.127464  3.186070  3.336985  2.277201  2.555143  3.849727
[43]  2.608764  3.807118  4.570531  2.572299  3.867606  5.675037  3.418622
[50]  3.678789  3.026411  1.843596  1.575882  3.406466  2.849537  3.661601
[57]  2.755899  3.295613  3.409166  3.170978

```

```

z <- cbind(x = x, y = rev(x))
plot(z)

```



Now we can run `kmeans()` on this input `z` and see what the results look like.

```

km <- kmeans(z, centers = 2)
km

```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-2.936304	3.127091
2	3.127091	-2.936304

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Within cluster sum of squares by cluster:

```
[1] 60.77499 60.77499
(between_SS / total_SS = 90.1 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

\$names

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

\$class

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “component” of your result object details cluster assignment/membership?

```
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

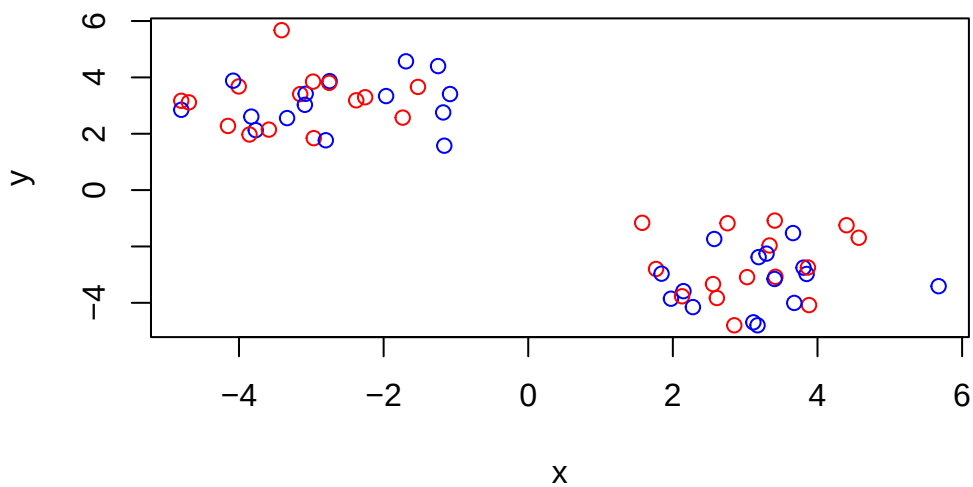
Q. What “component” of your result object details cluster center?

```
km$centers
```

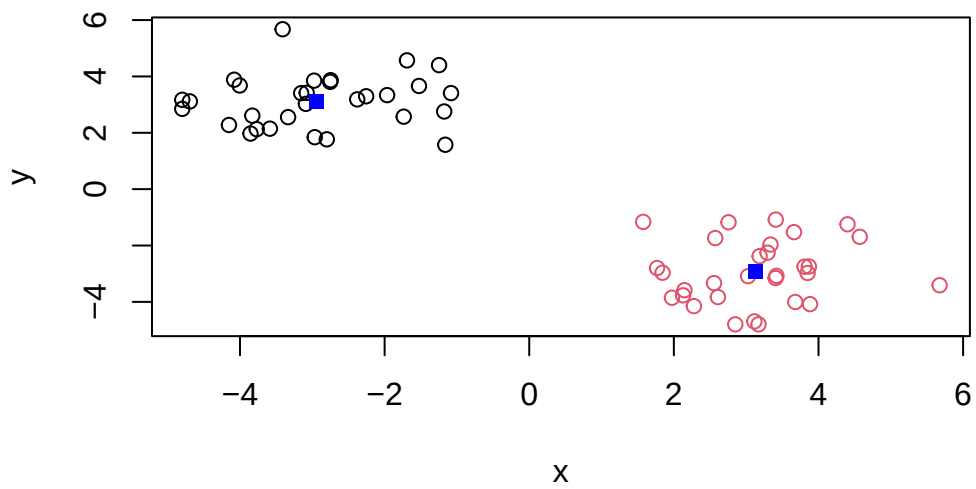
	x	y
1	-2.936304	3.127091
2	3.127091	-2.936304

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points.

```
plot(z, col = c("red", "blue"))
```



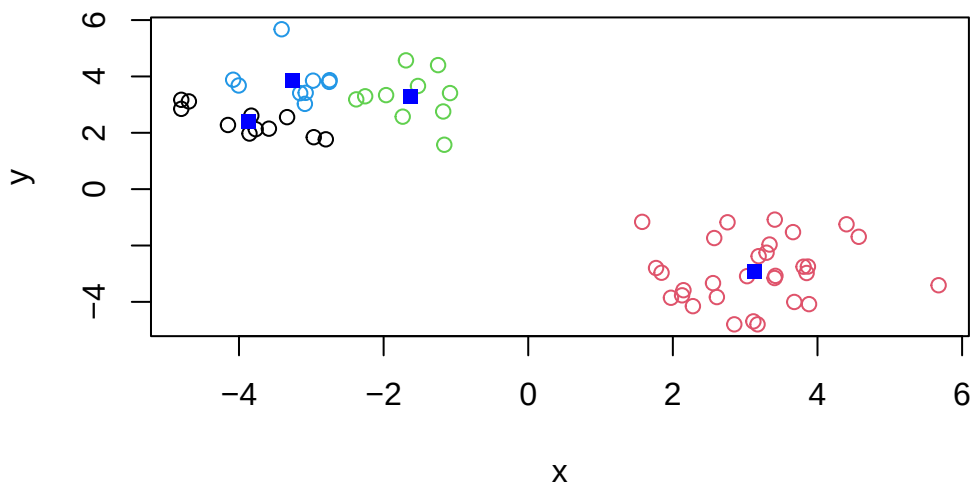
```
plot(z, col = km$cluster)
points(km$centers, col = "blue", pch = 15)
```



Q. Run a k-means clustering and plot the results asking for 4 clusters.

```
km4 <- kmeans(z, centers = 4)

plot(z, col = km4$cluster)
points(km4$centers, col = "blue", pch = 15)
```

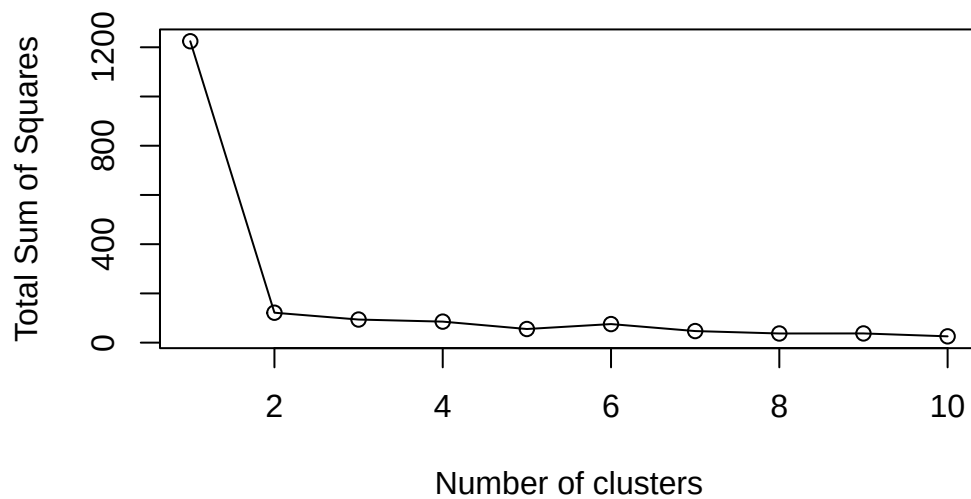


N.B. You need to tell K-means the number of clusters (i.e. set `centers = 2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for(i in 1:10) {
  km <- kmeans(z, centers = i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, typ = "o",
      xlab = "Number of clusters",
      ylab = "Total Sum of Squares")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)

hc
```

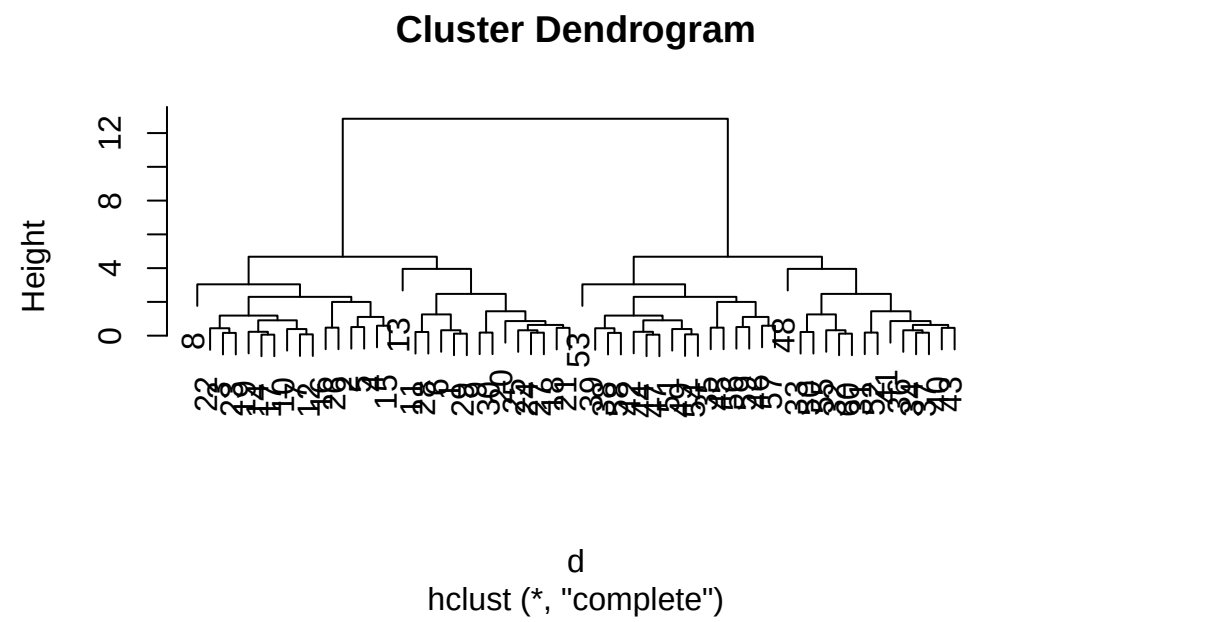
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.


```
plot(hc)
```



Once we have our `hclust()` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h = 8)
```

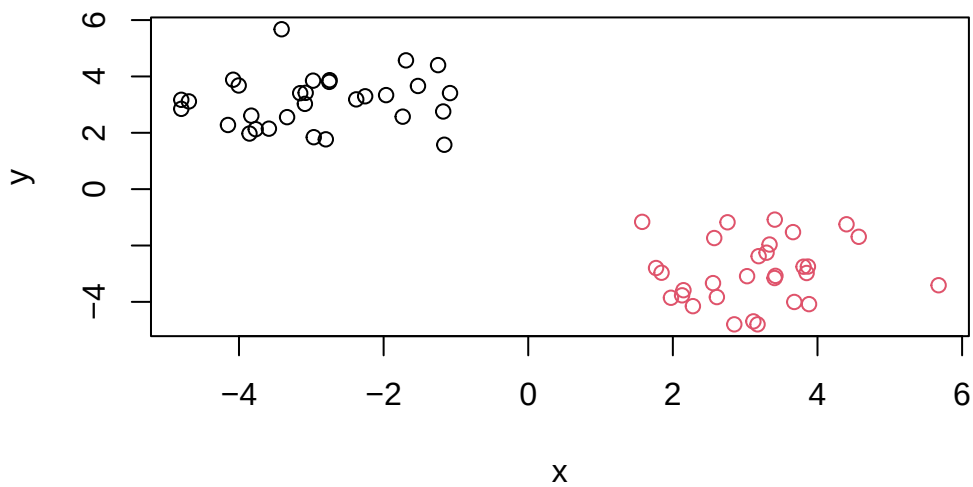
[illegible]

```
cutree(hc, k = 4)
```

[1] 1 2 2 2 2 1 2 2 1 2 1 2 1 2 2 2 2 1 2 1 1 2 2 1 1 2 1 1 1 3 3 3 3 4 3 3
[39] 4 3 3 4 3 4 4 4 4 3 4 3 4 3 4 4 3 4 4 4 4 3

Q. Make a plot of **z** with your **hclust()** results (i.e. colored by cluster membership).

```
grps <- cutree(hc, k = 2)
plot(z, col = grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK Food Data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns that have some regions being distinct from others.

Data Import

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

		X England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267

3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

```
## Preview the first 6 rows
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

```
# Note how the minus indexing works
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66

Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 4
```

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

```
read.csv(url, row.names = 1)
```

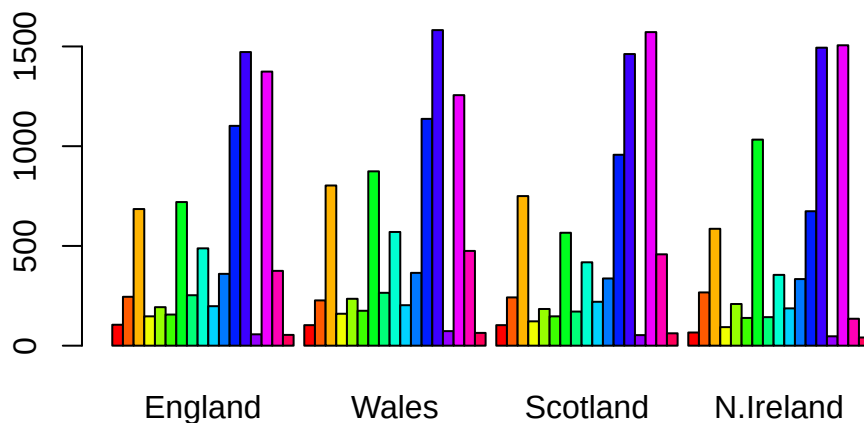
	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494

Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

Exploratory analysis

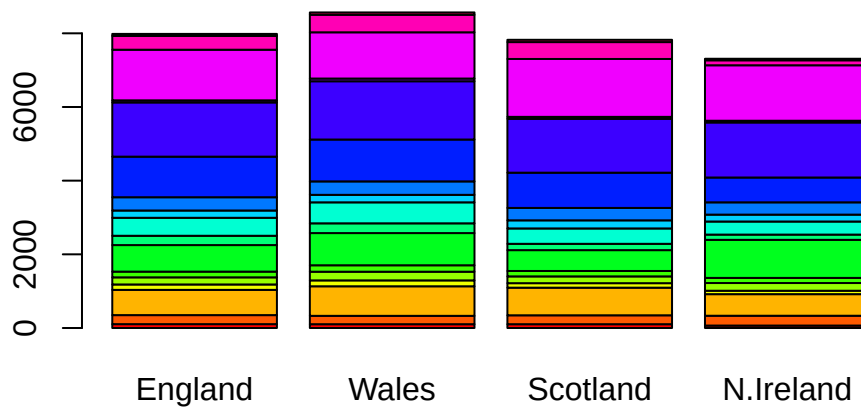
Make some plots to help make sense of obvious trends...

```
# Using base R
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3. Changing what optional argument in the above barplot() function results in the following plot?

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



Tidy the data

Fix anything that

```
library(tidyr)
```

```
# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

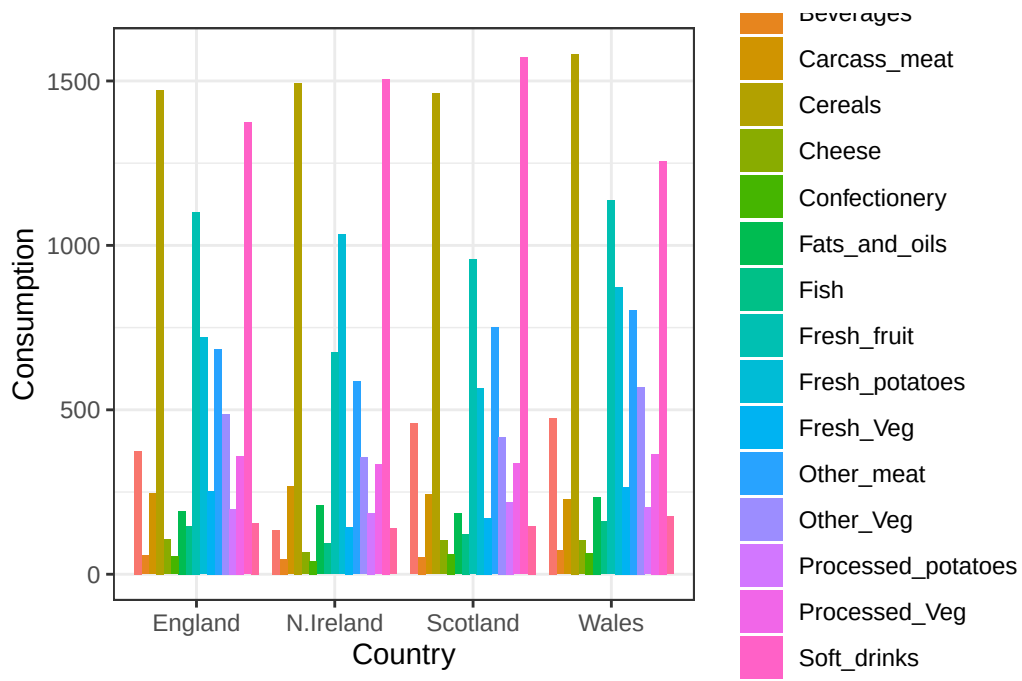
```
head(x_long)
```

```
# A tibble: 6 x 3
```

	Food <chr>	Country <chr>	Consumption <int>
1	"Cheese"	England	105
2	"Cheese"	Wales	103
3	"Cheese"	Scotland	103
4	"Cheese"	N.Ireland	66
5	"Carcass_meat "	England	245
6	"Carcass_meat "	Wales	227

```
# Create grouped bar plot
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4. Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

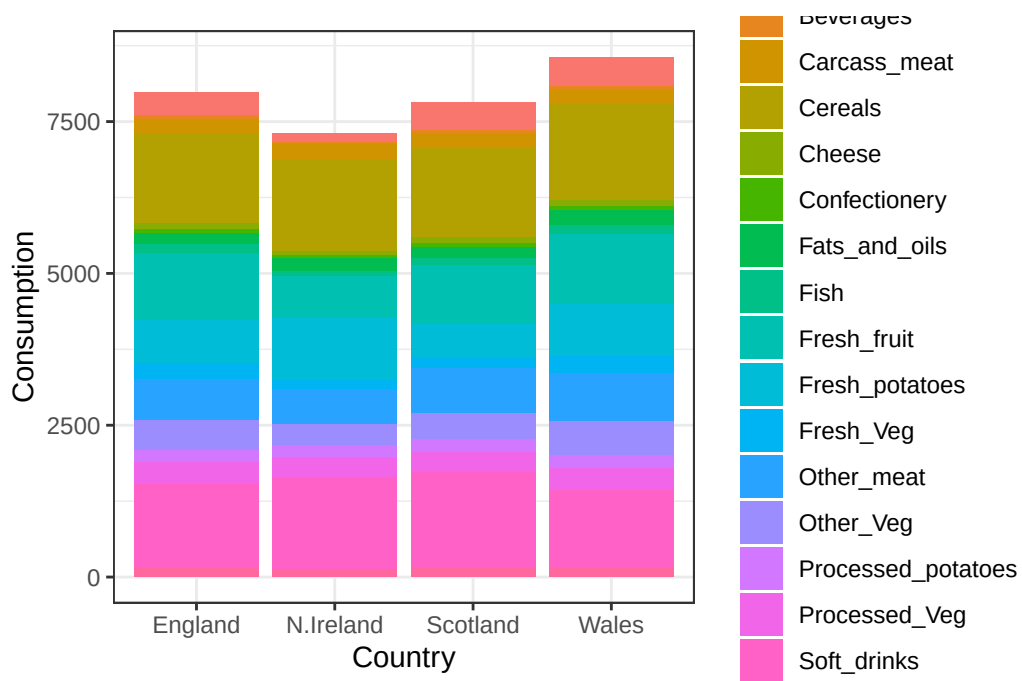
```
library(ggplot2)

ggplot(x_long) +
```

```

aes(x = Country, y = Consumption, fill = Food) +
geom_col(position = "stack") +
theme_bw()

```

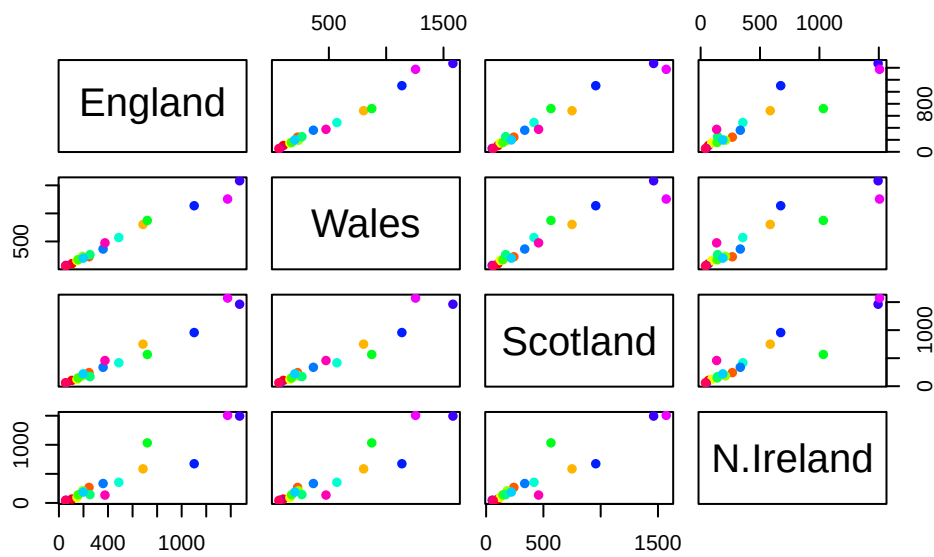


Q5 We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```

pairs(x, col=rainbow(nrow(x)), pch=16)

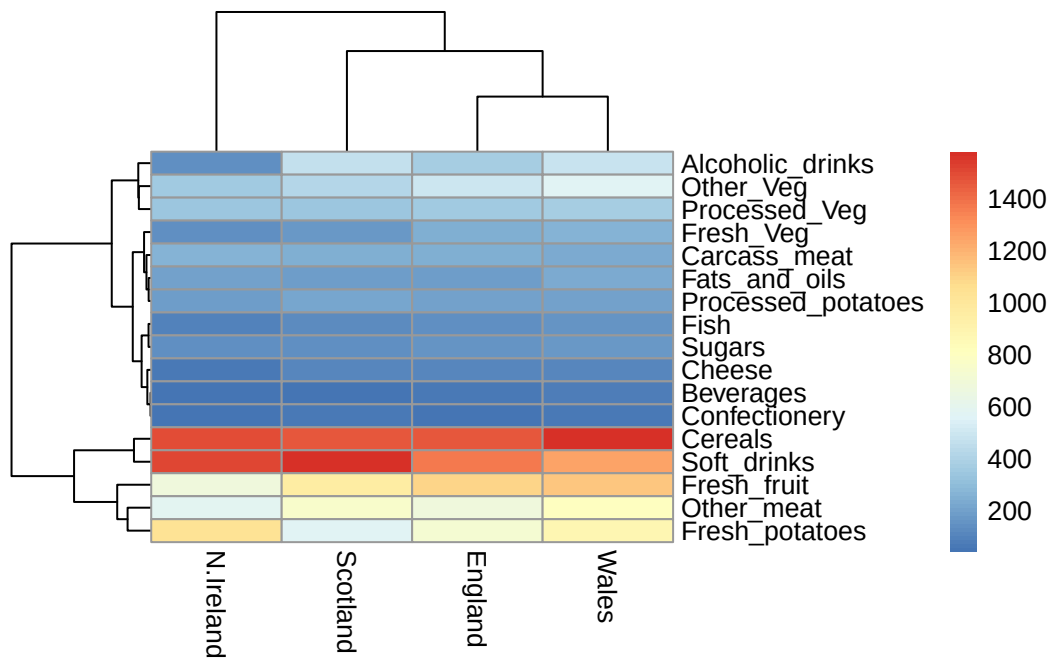
```

Points on the diagonal mean the variable is being plotted against itself, so they show a perfect correlation.

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set? England, Wales, and Scotland cluster together. This suggests they have similar food consumption patterns. Northern Ireland is separated, meaning its diet differs. The main difference is higher consumption of items like fresh potatoes and soft drinks in Northern Ireland.

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object.

```
# Use the prcomp() PCA function
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

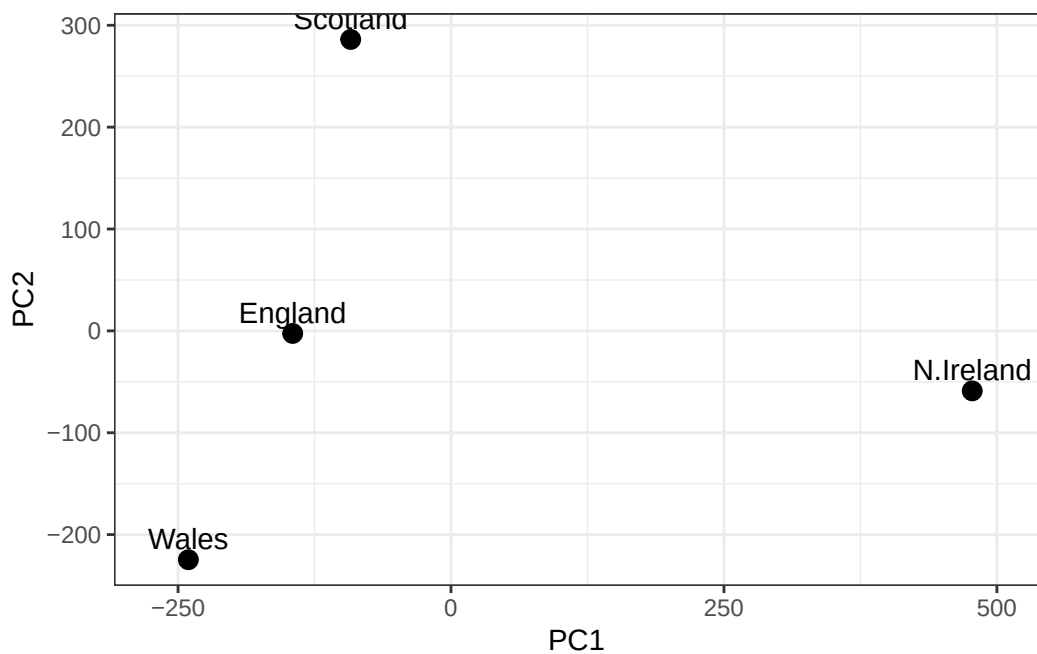
PC1 PC2 PC3 PC4

Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

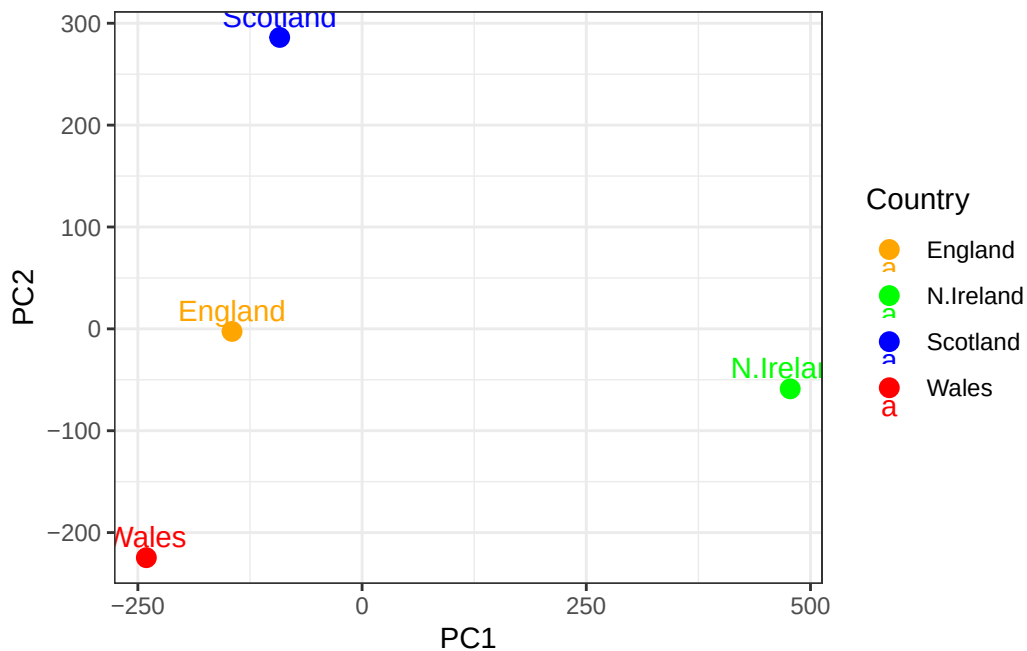
```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, color = Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  scale_color_manual(values = c(
    "England" = "orange",
    "Wales" = "red",
    "Scotland" = "blue",
    "N.Ireland" = "green"
  )) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

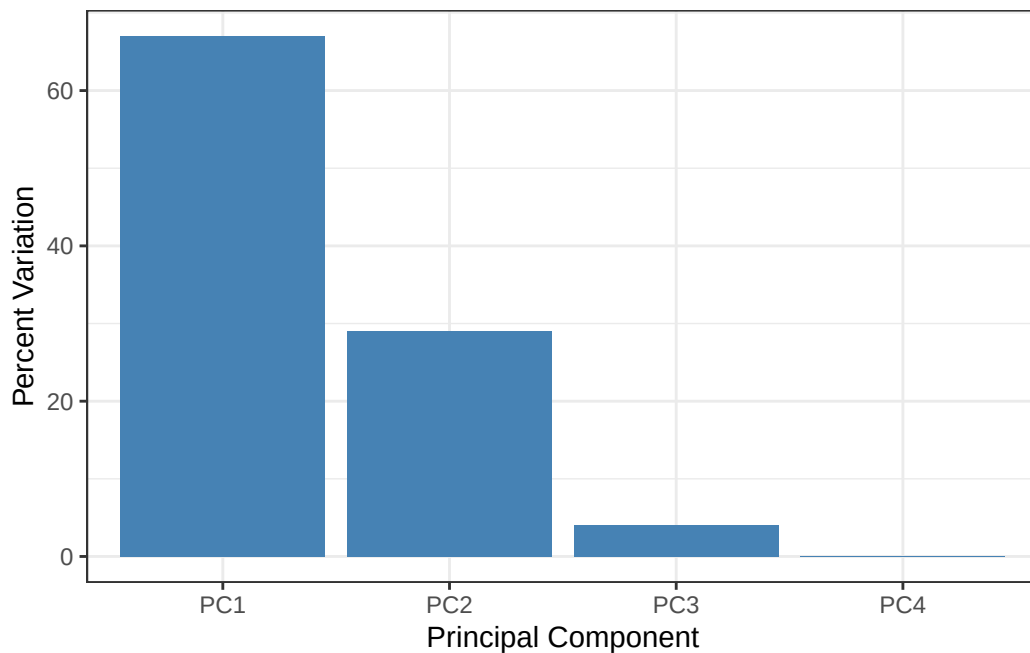
```
[1] 67 29 4 0
```

```
## or the second row here...
z <- summary(pca)
z$importance
```

	PC1	PC2	PC3	PC4
Standard deviation	324.15019	212.74780	73.87622	2.921348e-14
Proportion of Variance	0.67444	0.29052	0.03503	0.000000e+00
Cumulative Proportion	0.67444	0.96497	1.00000	1.000000e+00

```
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```

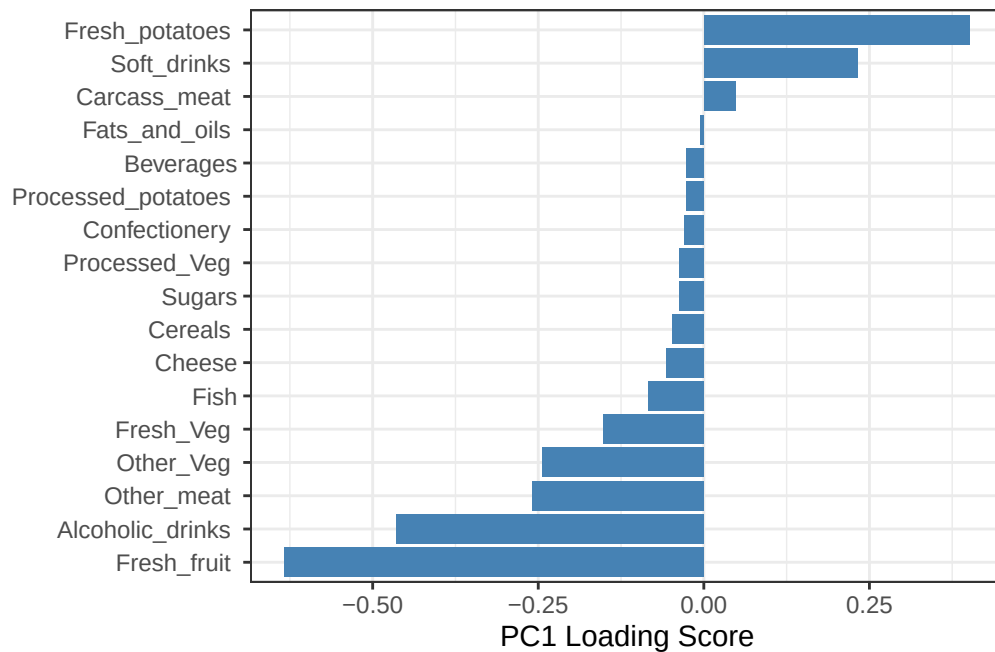


```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
```

```

y = reorder(rownames(pca$rotation), PC1)) +
geom_col(fill = "steelblue") +
xlab("PC1 Loading Score") +
ylab("") +
theme_bw() +
theme(axis.text.y = element_text(size = 9))

```

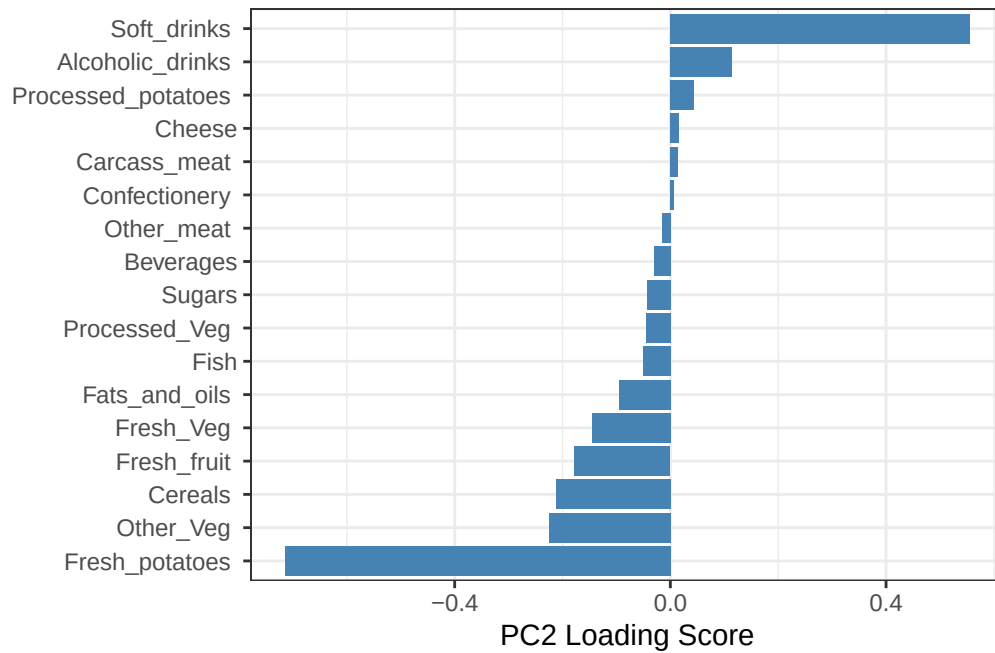


Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```

ggplot(pca$rotation) +
aes(x = PC2, y = reorder(rownames(pca$rotation), PC2)) +
geom_col(fill = "steelblue") +
xlab("PC2 Loading Score") +
ylab("") +
theme_bw()

```



PC2 mainly reflects differences in types of vegetable consumption, especially between fresh and processed foods. It separates countries based on whether they consume more fresh vegetables versus processed vegetables.